Axis Max Life

Configurability as an architectural requirement — when “no-code” is a KPI, not a feature

A client that specifies change velocity in days, across ten distribution channels and 125,000 sellers, has not asked for a configuration screen. It has specified an architecture — and most platforms fail the test in the same four places.

1–2	<1	~0	12	10+	6
days to configure a change	week to modify a workflow	change requests targeted	configurability domains	channels, each with its own rules	lifecycle stages, all configurable

CHANGE-VELOCITY TARGETS · AS SPECIFIED BY THE CLIENT

Regulatory and operational change absorbed through configuration, not engineering · time to market from three months to days

Aditya Singh · Architecture analysis authored for solution design · 2026

Vendor, platform and incumbent names removed. Targets are the client's; the architecture assessment and the four-level model are mine.

EXECUTIVE SUMMARY

A change-velocity target is an architecture specification — and it is failed far more often by the approval path than by the platform

Situation

The client specifies configuration in one to two days, workflow modification in under a week, near-zero change requests in business-as-usual, and time to market moving from three months to days.

These sit in the requirements document as business measures of success, alongside uptime and response time.

Complication

Read as a feature request, they invite the usual answer: an administration console and a screenshot of a form builder. Every serious platform can show that.

Read literally, they are far more demanding. They require configurability at four distinct levels, and they require an approval path that can keep pace — which is where the target is actually missed, because a change that takes two days to configure and six weeks to approve took six weeks.

The argument

Treat the velocity target as the architecture requirement it is. Specify which of the four levels each domain needs, design the state machine so channel variation is data rather than code, and make the change-approval path part of the configurability design.

Otherwise the platform passes the demonstration and fails the KPI.

THE NUMBERS THAT MATTER

4	12	6	10+	1–2	1
levels of configurability	domains in scope	lifecycle stages	channel variants	days, the stated target	failure mode nobody designs for

The central claim: configurable but unapprovable is not configurable. A platform that can change a rule in an afternoon, inside a governance process that takes six weeks to authorise it, has delivered a demonstration rather than a capability.

Source: Author's architecture analysis

HOW TO READ THIS DECK

The velocity targets and domain list are the client's; the four-level model and the assessment against it are mine

REQUIREMENTS

Client context

Change-velocity targets and the configurability domains as specified in the client's requirements pack.

- 1–2 day configuration target
- Sub-one-week workflow change
- Near-zero change requests in BAU
- Twelve configurability domains
- Six-stage lead lifecycle
- Ten-plus channel variants

ARCHITECTURE ANALYSIS

My contribution

The four-level configurability model, the domain assessment and the approval-path argument.

- Four-level configurability model
- Level required per domain
- State-machine design position
- Approval-path analysis
- Fit assessment against the model
- Applied to solution design

JUDGEMENT

Assessed

Level assignments per domain are an architect's judgement from the stated requirements, not a measured benchmark.

- Level assignment per domain
- Effort implication by level
- Fit ratings
- Sensitive to final requirements
- Not a product certification
- Method transfers unchanged

OUT OF SCOPE

Not claimed

Deliberately excluded. This is an architecture position, not a product claim or a delivery record.

- Any product's certified capability
- Delivery of this programme
- Any commercial position
- The incumbent's architecture
- Benchmark data against competitors
- Whether the deal was won

Source: Client requirements (context) · author's architecture analysis (models and assessment)

THE REQUIREMENT

The client specified change velocity the way it specified uptime — as a measure of success with a number attached, not as a feature to demonstrate

1–2 days
to configure a change

< 1 week
to modify a workflow

~0
change requests in business-as-usual

3 months → days
time to market for a new journey

Why the client wrote it this way

- Insurance distribution changes constantly — regulatory updates, channel restructures, incentive redesigns, new product launches.
- On the incumbent platform each of those was a vendor change request with a fee and a queue position.
- The target is not a technical aspiration. It is a direct statement of what went wrong last time.

What most responses do with it

- Answer with an administration console and a form-builder screenshot, which every serious platform has.
- Score well in the demonstration, because a demonstration only tests whether the screen exists.
- Then miss the target in production, because the screen was never the constraint.

What taking it literally requires

- Configurability specified at four distinct levels, with the required level named per domain.
- Channel variation held as data in a state machine, not as branches in code.
- A change-approval path designed to the same target as the change itself.
- Acceptance criteria that measure elapsed time to production, not availability of a screen.

The test that separates a real answer from a demonstration

Ask any platform to add a new lead source, visible to two of ten channels, with its own deduplication rule and its own disposition set — and then ask how long it takes to reach production, including approvals, regression and release.

The form builder answers the first half in minutes. The second half is where the one-to-two-day target is won or lost, and it is almost never in the demonstration script.

Source: Client requirements and stated change-velocity targets · author's analysis

THE MODEL

Four levels of configurability — and a platform that stops at level two will pass the demonstration and fail the target

LEVEL 1

Data configuration

Values, lists, masters and reference data changed through an administration screen.

EXAMPLE

Add a disposition reason. Add a branch. Change an escalation threshold.

Business administrator, no release

LEVEL 2

Structure configuration

Fields, forms, layouts and validation rules defined through a designer rather than written.

EXAMPLE

Add a field to the lead form, mandatory for one channel and hidden for another.

Business administrator, no release

LEVEL 3

Behaviour configuration

Workflow, state transitions, allocation logic and eligibility rules expressed as configuration.

EXAMPLE

Change allocation from round-robin to capacity-weighted for one channel only.

Configurator with review, controlled release

LEVEL 4

Composition

Whole journeys, sources and channel variants added as instances of existing patterns rather than new builds.

EXAMPLE

Add an eleventh distribution channel with its own hierarchy and journey, without code.

Configurator with design review

Where platforms actually stop, and why it matters here

Almost every enterprise platform delivers levels one and two convincingly. A good number reach level three for simple workflows and then require engineering for anything involving allocation logic or conditional state transitions.

Level four is rare, and it is precisely what a ten-plus-channel distribution estate needs. If adding an eleventh channel is a project rather than a configuration, the near-zero change-request target cannot be met — not because the platform is weak, but because the requirement was assessed at the wrong level.

Source: Author's architecture analysis

DOMAIN ASSESSMENT

Twelve domains, each needing a different level — naming the level per domain is what turns a vague “no-code” requirement into something a supplier can be held to

Configurability domain	Level	What changes without a release	Change frequency
Lead capture and creation	4	New sources, per-channel visibility, capture forms	Monthly
Activity and task management	4	Activity types, templates, attributes, statuses	Monthly
Workflow and journeys	4	State transitions, per-channel journey variants	Quarterly
Role, access and visibility	3	Hierarchy, data scoping, field-level permissions	Continuous
Geo and location controls	3	Geofence radius, mandatory tagging, spoof checks	Quarterly
Compliance and risk	3	Consent capture, retention, mandatory evidence	Regulatory
Communication and outreach	3	Templates, channels, throttles, suppression	Continuous
Notifications and alerts	2	Triggers, recipients, escalation ladders	Monthly
Incentives and performance	3	Targets, scoring rules, leaderboard definitions	Quarterly
Reporting and analytics	2	Report definitions, exports, scheduled feeds	Continuous
Admin panel and masters	1	Reference data, branches, products, reasons	Continuous
System and integration	2	Endpoints, mappings, retries, throttling	Rare

Source: Client requirements and stated change-velocity targets · author's level assessment

Four domains need level four and four more need level three — which means eight of twelve require configurability beyond what a form builder provides. That distribution, not the domain

THE CORE MECHANISM

A six-stage lead lifecycle held as a configurable state machine — channel variation becomes data, and adding a channel stops being a project

01	02	03	04	05	06
Capture and sources	**Deduplicate and validate**	**Allocate and own**	**Journey and disposition**	**Age and recycle**	**Consent and closure**
Sources enabled or disabled per channel; form and field builder with channel, sub-channel and role visibility	Match rules on mobile, email or tax identity, each with hard-block, soft-warn or flag behaviour per source	Rule-engine allocation on geography, branch and role — round-robin, capacity or performance-weighted, with audited override	Per-channel journey variants, disposition masters, ownership locks and anti-cherry-picking controls	SLA timers, auto-recycle thresholds and re-allocation rules defined per lead type	Closure and drop-reason masters, consent state, suppression and retention rules

Why a state machine, specifically

- Ten channels do not need ten journeys. They need one journey whose transitions, rules and vocabularies are parameterised.
- Every difference between channels becomes a row in configuration rather than a branch in code.
- An eleventh channel is then an instance of an existing pattern, which is the definition of level-four configurability.

What has to be true for it to work

- The state set must be closed and shared — a channel that needs a private state has broken the model.
- Dispositions must be mastered centrally, or reporting fragments the moment a channel adds its own.
- Allocation must be a rule engine, not a set of coded strategies with a dropdown in front of them.

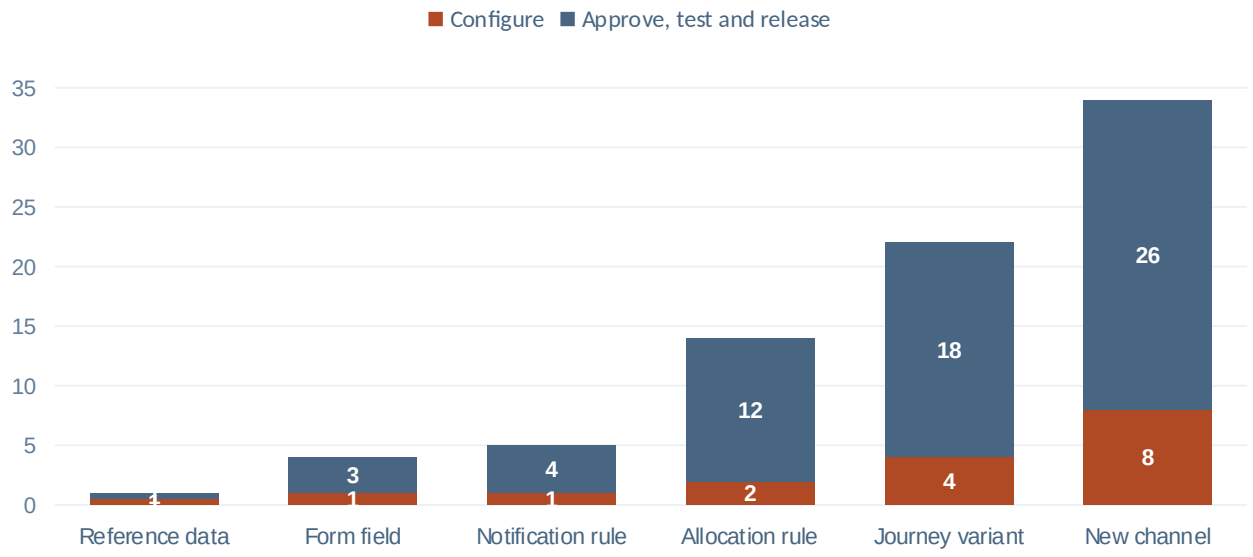
The failure mode to watch

- The first genuinely awkward channel requirement arrives under delivery pressure and is met with a code branch.
- That single exception ends the model — every subsequent channel gets its own branch because the precedent exists.
- Refusing that first fork is a governance decision, not a technical one, and it usually has to be made at the worst moment.

Source: Client requirements and stated change-velocity targets · author's analysis

THE FAILURE NOBODY DESIGNS FOR

A change configurable in two days and approvable in six weeks took six weeks — the approval path is part of the configurability requirement



Illustrative elapsed days by change type — configuration effort against everything that follows it

Where the target is actually lost

- Configuration effort is small and roughly linear. Approval and release effort grows with blast radius and dominates everything above a trivial change.
- The stated one-to-two-day target is achievable for the left-hand changes and unreachable for the right-hand ones under any conventional release process.
- The honest answer is not that the platform is slow. It is that the change-approval path was never designed to the same target.

What the design has to include

- Change classes with pre-agreed approval paths, so low-risk configuration does not queue behind releases.
- Automated regression scoped to the blast radius of the change, not to the whole platform.

Why this belongs in the architecture, not the operating model annexe

The change classes and their blast radii are determined by how the platform is built. If allocation logic and journey variants are genuinely data, their blast radius is small and a fast approval path is defensible to risk and audit. If they are code with a configuration screen in front of them, no governance design can safely approve them quickly.

That is why the approval path cannot be designed after the platform. The architecture decides what is safely approvable, and therefore what velocity is achievable at all.

Source: Author's analysis — elapsed-day figures are illustrative

FIT AGAINST THE MODEL

Assessed level by level, the honest position is strong at one to three, real work at four, and an open question on the approval path

Capability	What it means here	Fit	Status
Level 1 — data configuration	Masters, reference data, reasons and thresholds via administration	NATIVE	Met
Level 2 — structure configuration	Field, form and layout designers with role and channel visibility	NATIVE	Met
Level 3 — behaviour configuration	Workflow, transitions, allocation and eligibility as rule configuration	NATIVE	Met
Level 4 — composition	New channels and journeys as instances of existing patterns	BUILD	Partly
Channel variation as data	One state machine, ten-plus parameterised variants, no per-channel branch	CONFIGURABLE	Met
Blast-radius-scoped regression	Automated test selection driven by what the change touches	BUILD	Gap
Change classes and approval paths	Pre-agreed authority per class — a joint design with the client's risk function	GAP	Open

How I would present this to the client

Not as a scorecard. The first five rows are a credible answer to the stated requirement and should be shown as such, with the level-four work priced honestly rather than claimed away.

The last two rows are more useful as a joint design question than as gaps. Blast-radius-scoped regression and pre-agreed change classes cannot be delivered by a supplier alone — they need the client's risk and release functions in the room. Proposing that workshop in the bid, rather than discovering the need in month four, is the difference between meeting the velocity target and explaining why it was never achievable.

Source: Author's architecture analysis

WHAT TRANSFERS

Four rules for treating configurability as an architecture requirement

01	**Name the level, not the word**	“No-code” is unenforceable and every platform claims it. Specifying which of the four levels each domain requires makes the requirement contractible and testable at acceptance.
02	**Make channel variation data, never code**	One state machine with parameterised variants means the eleventh channel is configuration. One code branch per channel means it is a project — and the first branch sets the precedent for all the rest.
03	**Design the approval path to the same target**	A change configurable in two days and approvable in six weeks took six weeks. The velocity target belongs to the whole path to production, not to the configuration step.
04	**Test with elapsed time, not with a screen**	Acceptance should measure how long a representative change takes to reach production including approval and regression — not whether an administration screen exists.

Configurable but unapprovable is not configurable. The architecture decides what can safely be approved quickly, which means it decides the achievable change velocity — long before any governance design is written.

Source: Author's architecture analysis